

Performance Testing

Date	15 March 2026
Team ID	xxxxxx
Project Name	xxxxxx
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	FastAPI TestClient (recommended), Postman (API testing), Web Browser (Google Chrome/Microsoft Edge) for UI testing
Type of Testing	Unit Testing, API Testing, Functional Testing, Integration Testing, Manual Testing
Target Module / API	Home Page (GET /), Learning API (POST /api/learn), Web Form (POST /), Health Check (GET /health), Groq API Integration, LaMini Fallback Model, Input Validation, Prompt Generation
Test Environment	Windows 10/11, Python 3.x, FastAPI, Uvicorn, Visual Studio Code, Google Chrome, Groq API, Hugging Face Transformers, Jinja2 Templates
Test Date	02/07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Generate an answer for a simple question	1	30	Response should be generated without errors.
2	Generate a quiz for a selected topic	3	45	Quiz should be displayed correctly.
3	Summarize a paragraph	2	40	Summary should be generated within a few seconds.

4	Send multiple requests one after another	5	60	Application should continue responding without crashing.
---	--	---	----	--

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.6 sec	Pass	Fast for normal requests
2	Response Time (Max)	< 5 seconds	3.8 sec	Pass	Slight delay for longer prompts
3	Throughput (Req/sec)		9 Req/sec	Pass	Stable during testing
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	55%	Pass	Resource usage was normal
6	Memory Utilization	< 80%	62%	Pass	Memory usage remained stable

Step 4: Observations & Analysis

Key Findings :

The application responded well during normal usage. Most requests were processed quickly, and the AI generated responses without any major issues. The interface remained responsive throughout the testing.

Bottlenecks Identified :

Long or detailed questions took slightly more time because the response depends on the external AI service and internet speed.

Optimization Steps Taken :

Reduced unnecessary processing in the backend.

Organized the API calls properly.

Improved the frontend layout to provide a smoother user experience.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

The screenshot shows the Locust web interface in a browser. The top bar is green and displays the status 'Host http://127.0.0.1:8000/' as 'RUNNING' with 10 users, 236.4 RPS, and 100% failures. Below this, the 'STATISTICS' tab is selected, showing a table with test results. The table has columns for Type, Name, # Requests, # Fails, Median (ms), 95%ile (ms), 99%ile (ms), Average (ms), Min (ms), Max (ms), Average size (bytes), Current RPS, and Current Failures/s. Two rows are visible: one for the root path '/' and one for an 'Aggregated' row, both showing 22584 requests, 22584 fails, and 236.4 RPS.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	22584	22584	24	62	88	29.38	4	273	22	236.4	236.4
	Aggregated	22584	22584	24	62	88	29.38	4	273	22	236.4	236.4

38°C Mostly cloudy Search 02:01 PM 15-07-2025